Native Functions

Host implementations as ordinary function values

Gregory S. DeLozier, PhD

Another Useful Operation

What must change to add square root?
A new parser case for every library operation would keep growing the language implementation.

Two Languages, Two Jobs

Language	Job
Vertex	The program our interpreter runs
Python (the host)	The interpreter and its library implementations

Code examples identify which language executes them.

The Host Registry Is a Python Dictionary

Python host

```
  # code/native_functions.py (excerpt)
NATIVE_FUNCTIONS = {
    "math.sqrt": sqrt,
    "math.hypot": hypot,
}
```

String key on the left; Python callable on the right.

Creating a Native Function

Vertex program

```
  sqrt = native("math.sqrt", ["x"]);
print(sqrt(9));
```

Prints `3.0`.

Names, Parameters, and Arity

Vertex program

```
  distance = native("math.hypot", ["a", "b"]);
print(distance(3, 4));
```

Prints `5.0`.

Construction and Invocation

Expression	Work performed
`native("math.sqrt", ["x"])`	Look up the Python registry; construct a function
`sqrt(9)`	Pass 9 to the stored Python callable

The Constructor Is a Value

Vertex program

```
  connect = native;
sqrt = connect("math.sqrt", ["value"]);
print(sqrt(36));
```

Prints `6.0`.

Passing Native Functions

Vertex program

```
  function apply(f, x) {
     return f(x);
};
sqrt = native("math.sqrt", ["x"]);
print(apply(sqrt, 25));
```

Prints `5.0`.

Records and Closures

Vertex program

```
  function makeOperations() {
     sqrt = native("math.sqrt", ["x"]);
     return {"root": sqrt};
};
ops = makeOperations();
print(ops.root(49));
```

Prints `7.0`.

Python Registers the Implementation

Python host

```
  from native_functions import register_native

def combine(a, b):
    return a + b

register_native("combine", combine)
```

Python stores a callable under the key `"combine"`.

Vertex Obtains and Calls It

Vertex program

```
  f = native("combine", ["a", "b"]);
print(f(2, 5));
```

Prints 7, after the Python registration.

A Registry Key Is Just a Name

Name or value	Meaning
`"math.sqrt"`	A string key in the Python registry
Python `sqrt`	The registered implementation
Vertex `root`	A variable holding a native function value

No Python import or source evaluation happens in `native`.

Shared Call Evaluation

1. Evaluate the callee.
2. Evaluate arguments left to right.
3. Check the declared arity.
4. Run the Vertex body or invoke the host.

Crossing the Value Boundary

Result	Accepted?
Null, boolean, number, string, function	Yes
Lists and objects of Vertex values	Yes
Python tuple or datetime instance	Needs an adapter

Errors Keep Their Meaning

An unknown registration fails at construction.
Wrong arity fails before the host invocation.
Python exceptions propagate to the embedding host.

Function Identity

Vertex program

```
  f = native("math.sqrt", ["x"]);
g = native("math.sqrt", ["x"]);
print(f == g);
print(copy(f) == f);
print(type(f));
```

Prints `false`, `true`, then `function`.

A Portable Interface

Vertex source identifies an operation.
The interpreter connects it to its host implementation.
Python and Go hosts can implement the same contract.

Practice

Vertex: call a native square-root function twice.

Vertex program

```
  function applyTwice(f, x) {
     return f(f(x));
};
sqrt = native("math.sqrt", ["x"]);
print(applyTwice(sqrt, 81));
```

Expected result: `3.0`.

The Next Unit: Modules

A file can define Vertex functions, native functions, and data.
What should importing that program return?